Why every timed IC row of run 39376405326 crashed (diagnosis, no timing)

2026-10-20

Why every timed IC row of run 39376405326 crashed (diagnosis, no timing)

Finding. In v2's `arm.sage`, `relation_digests` built its third digest from a dictionary default written as the literal `0`. The Sage preparser rewrites every integer literal in a `.sage` file as a Sage `Integer`, so the line

```
row[col] = (row.get(col, 0) + sg * pow(int(lam), int(e), int(p))) % int(p)
```

ran as `row.get(col, Integer(0))`. An `Integer` plus an `int` is an `Integer`, and an `Integer` modulo an `int` is too. Every value of `rows` was therefore a Sage `Integer`. `Integer` is not an `int` subclass, so `json.dumps(rows)` raised `TypeError: Object of type Integer is not JSON serializable`. The row was never written, and the process exited with code 1.

This happened after the row's timed interval, in every B, N, Naa and NX process of all 16 points. The rho rows, S0, O1 and the field checks never call `relation_digests`, and they all passed.

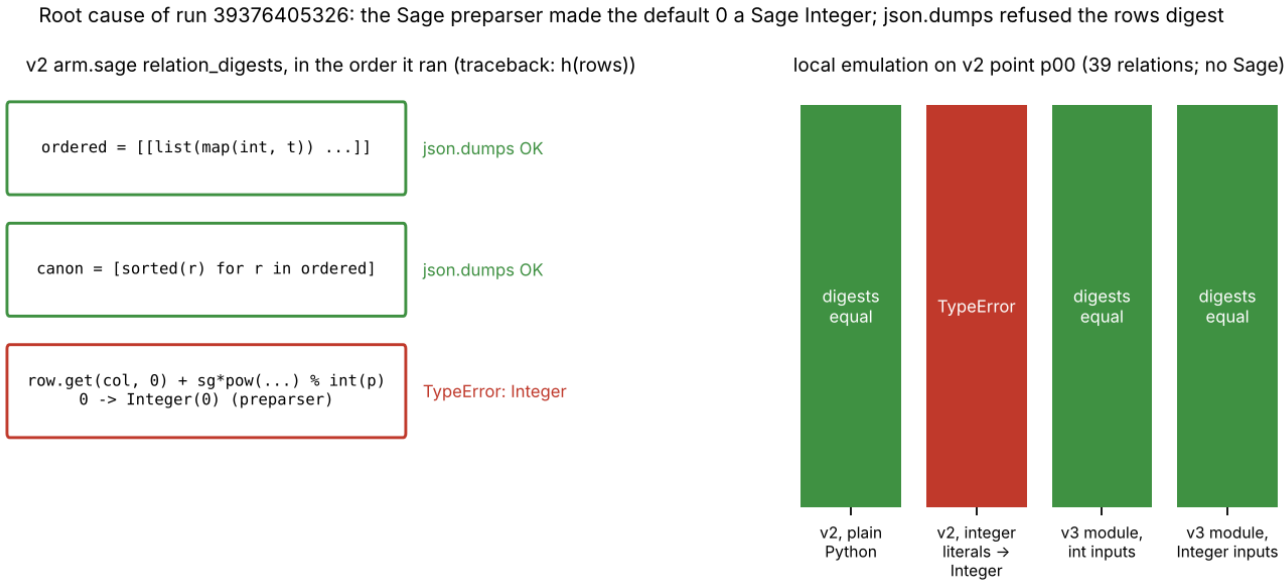


Figure 1: The failing digest, and the local emulation on v2 point p00. Source: `../make_figures.py crash_diagnosis`; data: `emulate_preparser.json`; vector: `../figures/crash_diagnosis.svg`.

Evidence

1. **The archived traceback** (`run_github_39376405326/panel/logs/p15-B.err` on branch `claude/n19-native-solve`, PR #1003). Every one of the 64 IC rows ends in `relation_digests`, with the caret under `h(rows)` in the

line `return {'ordered_sha256': h(ordered), 'canonical_sha256': h(canon), 'rows_sha256': h(rows), ...}`. Python evaluates those values left to right. `h(ordered)` and `h(canon)` therefore succeeded, and only `rows` held a non-JSON value.

2. **Static reading.** `ordered` is built with `list(map(int, t))`, and `canon` sorts it. In `rows`, `sg`, `pow(int, int, int)` and `int(p)` are all Python ints. The only other operand is the default `0`, which is a Sage `Integer` in a `.sage` file. A relation never repeats a column in practice, so every row value starts from that default and is an `Integer`.
3. **Local emulation, no Sage** (`emulate_preparser.py` → `emulate_preparser.json`).
   - The script runs v1's pure-Python tauDlp replay with the v2/v3 binding on v2 point p00 (the scalar and rng seed of the crashed rows p00-B and p00-N). This gives 39 relations, and the log verifies.
   - It takes v2's `relation_digests` verbatim from v2 `arm.sage` with `ast`:
     - Compiled as plain Python, the function succeeds. In plain Python the literal is an int, which is why v2's local checks, all pure Python, could not see the bug.
     - Compiled again with the preparser's integer-literal rule (`n` → `Integer(n)`, with a stand-in `Integer` that has Sage's coercion), it raises exactly `TypeError: Object of type Integer is not JSON serializable`. The first two `json.dumps` calls succeed and the third (`rows`) fails. The failing object's leaves are `int` and `Integer`, and its first row is `[(15, Integer(30942))]`.
   - v3's `relation_digest.relation_digests` returns the same three digests as v2's plain-Python run, from both int and stand-in `Integer` inputs.
4. **Exact reproduction in the pinned image** (the v3 workflow, job `sage-smoke`, step "Reproduce v2's failing calls"; `digest_repro.sage`). This step reruns v2's own `arm.sage` from main for rows p00-N and p00-B, with run 39376405326's arguments from v2 `schedule.json`. The library is built from `lib.rs`, which is byte-identical to v2's. The step requires each call to exit 1 without a row file, with the traceback through `relation_digests` at `h(rows)`, ending in the TypeError above. In the same image it records `preparse()` of the v2 line, the types it produces, and v3's digest and strict converter on Sage `Integer`, `RealNumber`, `RealDoubleElement`, a GF(2^{19}) element and a rational. The step is recorded and does not gate the run. The v3 smoke gate is what decides whether timed rows run. Its result is in the run's `n19-native-tau6-v3-smoke-*` artifact (`diagnosis/digest_repro.json`).

Why v2's checks missed it

v2 check	ran `relation_digests`?	under the Sage preparser?
`smoke/local_v2.py` (O1 and the panel, pure Python)	no	no
`diagnosis/s0_v2_local.py` (no Sage)	no	no
untimed Sage diagnostic run 39329564781 (`sage_diag.sage`)	no	yes
S0 and O1 on the runner	no	yes
timed B, N, Naa and NX rows	**yes**	**yes**: crashed

No v2 check ran the arm path that writes an IC row under Sage before the timed segments.

The fix (v3) and the audit of other Sage-type hazards

- **The fix.** `relation_digests` moves to `../relation_digest.py`, plain Python that the preparser never touches. Its inputs go through `operator.index`, which is exact and refuses floats, and every value is built from Python ints and checked to be `int`. The digest definitions are unchanged: v2's `sorted(row.items())` and v3's list of `[c, v]` serialize to the same JSON. `arm.sage`, the verifier and the local checks import this one module.
- **Audit of every place where JSON or hashing meets a value in `arm.sage`.**

site	v2	risk	v3
`relation_digests` `rows`	`row.get(col, 0)`	**crash** (above)	moved to `relation_digest.py`, ints only
`relation_digests` `ordered` and `canon`	`int()` on every term	none	same, via `operator.index`
row output `json.dump(row, default=jplain)`	`jplain` turned non-integral values into `str()` silently (for example a rational or a field element)	silent wrong type	`relation_digest.plain`: Sage Integer → int, Sage reals → float, anything else **raises**, naming its path; no `default=` hook
`PLANT` counters, `stats` in S0	literal `0` → Integer	serialized through `jplain` (fine)	through `plain` (exact int)
`tauDlp` statistics (`log`, `gb_calls`, ...)	Sage Integer	through `jplain`	through `plain`
O1 comparison `jplain(st[f]) == a[f]`	Integer vs int	fine	`plain`
S0 `t % N_BITS`, `2 ** N_BITS`	Integer	`random.randrange` takes `__index__` (same draw)	wrapped in `int()`; v2's 300 systems are re-derived with v2's seed and gated
ctypes calls (`native_binding.py`)	`int()` on every argument	none	unchanged
`hashlib`	file bytes and JSON bytes only	none	adds the SHA-256 of each row's exact bytes (`raw_b64`) and of S0's detail (sorted-key JSON)
the verifier	plain Python	none	recomputes every relation digest from each row's raw relations with the same module

- **What exercises the real path now.** O1 computes the relation digests of all ten archived ECDLPs under Sage, before any timed row. The smoke job runs every arm (B, N, Naa, NX and rho) on two points, plus S0 and O1, in the pinned image, and the timed job needs it to pass.